

## Programming Assignment 5

### KnightNet

Due: 11/23/2025 at 11:59pm

*Objective: Students will apply concepts of graph algorithms in this programming assignment.*

#### Assignment Description:

Welcome to KnightNet, noble defender of the realm! You've been summoned by the Order of the Knights to design a covert surveillance network across the Kingdom of Pegasus. Your mission: connect a series of enchanted watchtowers and magical sensors scattered across the land to form a secure communication network.

But beware, laying magical fiber conduits is costly, and visibility to enemy sorcerers is a threat. The higher the visibility of a connection, the more likely it is to be intercepted. Your task is to build a Minimum Spanning Tree (MST) using Prim's algorithm, minimizing both cost and visibility exposure.

To further protect the realm, the Knights have deployed decoy towers, illusions meant to mislead enemy scouts. These towers appear in the network data but must not be included in the MST.

For this assignment, you must follow these **requirements**.

1. Create a public class called `KnightNet`.
2. The class constructor for `KnightNet` takes two parameters that will assist you in getting your network setup.
  - a. The first parameter is a `String` object that contains the name of a text file with all network connections the Kingdom of Pegasus has. NOTE: All text files are in the same directory as your driver file. Do not create any packages/subdirectories. Packages/Subdirectories used will result in your code throwing an exception which will fall under the runtime crashing grading criteria. Those points can't be disputed back.
    - i. Each text file contains a line representing an edge with the following format separated by a comma:  
`NodeA,NodeB,Cost,Visibility,IsDecoy`
    - ii. `NodeA` and `NodeB` are strings.
    - iii. `Cost` is an integer type.
    - iv. `Visibility` is an integer type.
    - v. `IsDecoy` is a `Boolean` type (true means decoy).
  - b. The second parameter is a primitive `int` that represents the maximum visibility.
3. Each text file contains a line representing an edge with the following format:  
`NodeA,NodeB,Cost,Visibility,IsDecoy`
4. You will need to implement a non-static method called `getRealNodes` that returns a `HashSet` of nodes that aren't considered decoys. A node is a decoy if it is part of an edge

that is considered a decoy. If your HashSet doesn't contain at least 2 nodes, then it is impossible to design a minimum spanning tree.

5. You will need to implement a non-static method called `computeMSTCost` that will return an integer value representing the smallest cost possible based on the given visibility constraint. The driver file will already select your starting node (first parameter). The second parameter represents the visibility constraint. This method will be called twice in the driver to show the original graph and the effect of a breached node. You are required to use Prim's algorithm for computing the minimum spanning tree. If you use a different algorithm, the graders will give no credit for this method, even if the output is correct.
6. Based on the resulting minimum spanning tree, you will need to display the list of edges that constructed the minimum spanning tree. You will implement a non-static method called `displayEdges`. This method will display each edge along with its associated cost and visibility. The edges must be displayed in a certain format in order to receive full credit.  
The format is shown as follows: `N1 - N17 | Cost: 5 | Visibility: 4`  
Each component has a single whitespace character. Along with the edge, you must display the source node with the lowest label first. For example, if you displayed "`N17 - N1`", then points would be deducted. Since there are multiple edges to be displayed from the resulting minimum spanning tree, the list of edges must be sorted by source nodes in ascending order. If there is a tiebreaker between the source nodes, then you must sort by destination nodes. Checkout the sample output to see examples.
7. You will need to implement a non-static method called `removeNode`. This method will physically remove the selected node from the graph. The parameter represents the selected node. The method does not return anything.
8. You may use (and it's recommended) helper classes that you can define in your `KnightNet.java` file.
9. You may create additional helper methods and attributes if needed as long as they are implemented and called in your solution file and NOT called from the driver file. Adding extra methods to call in the driver file will not match what the graders will use to evaluate your code. This will result in a low score with no changes to be applied!
10. Your code must run within 3 seconds on Eustis. If it runs longer than 3 seconds, you will not be eligible for any points in the rubric except for code compilation, program run, code style, code submission/header, and comments.

A driver file (`KnightNetDriver.java`) has been provided for you to show you how the methods are called along with 5 test cases. The text files are also provided for you that you will need to use in building the networks. The text file itself must be in the same directory as the runner file.

**What to submit:** Submit a file called `KnightNet.java` to webcourses. You are not required to submit the driver file as that will be provided for the graders to test your code. Please make sure the driver file provided works for your code. Any name changes may cause your program not to work when graded, which will result in a lower score on the assignment and would not be changed.

**Comment Header:** Please make sure to provide the appropriate comment header (like in the Eustis assignment) as the first thing on the top of your file. Otherwise, if your comment header is done incorrectly including not being placed literally at the top of the file will result point deductions in accordance with the rubric. They must be formatted as such and placed on the first line of the file with no whitespace above:

```
/* INSERT YOUR NAME HERE  
   Dr. Steinberg  
   COP3503 Fall 2025  
   Programming Assignment 5  
*/
```

**What happens if my code doesn't compile? How will my code be graded?** If your code falls under not compiling, you will not be eligible to receive any points on the test cases, method implementations (all three versions), categories. That means you would receive 0 points in those respective categories. The only full points you would be eligible for are code style, comments, and code submission/header if directions were followed. You will also receive no marks if your code does not compile.

**What happens if my code compiles, but crashes or throws an exception or is stuck in an infinite loop? How will my code be graded?** If your code compiles, but crashes or throws an exception or is stuck in an infinite loop, you will not be eligible to receive any points on the test cases, method implementations (all three versions), categories. That means you would receive 0 points in those respective categories. The only full points you would be eligible for are code style, comments, and code submission/header if directions were followed. You will also receive low marks if your code does compile.

**Important Note for running Eustis:** Many of you are probably using IDEs like Netbeans and Eclipse to build your Java Solutions. Please note that some of these IDEs will automatically place your Java file in some sort of package. Please make sure your Java file is not defined in some package as this can result package private errors. Any such error that occurs during the grading will not be fixed and points will be deducted as such in accordance with the respective categories in the rubric. Also, DO NOT create a main method in your solution file!! This will result in your code not running properly with the driver file which will result in points being deducted from the respective categories.

**Code Style Requirements:** It is expected that students will follow the provided code style guidelines. These guidelines are available in webcourses on the assignment page. Students who violate any of the guidelines will receive point deductions!

### **AI Syllabus Policy Reminder**

The following comes from our course syllabus.

*Use of Generative Artificial Intelligence (GenAI) is prohibited. Use of GenAI tools via website, app, or any other access, is not permitted in this class. All components of assignments in this course must be independently and originally completed by the student. Representing work created by GenAI as your own will be treated as plagiarism.*